

Лабораторная работа № 13

Программирование в командном процессоре ОС UNIX. Ветвления и циклы

Взваров Михаил, НПИбд-01-25

Цель работы

Изучить основы программирования в оболочке ОС UNIX/Linux и приобрести навыки написания более сложных командных файлов с использованием логических управляющих конструкций и циклов.

Задание

1. Написать командный файл с использованием `getopts` и `grep` для анализа параметров командной строки и поиска строк по заданному шаблону.
2. Написать программу на языке C, определяющую знак введённого числа и передающую результат через код завершения, а также командный файл, анализирующий значение `$?`.
3. Написать командный файл для создания N файлов `1.tmp`, `2.tmp`, ..., `N.tmp` и удаления всех созданных файлов.
4. Написать командный файл для архивирования файлов каталога с помощью `tar` и отбора файлов, изменённых менее недели назад, с использованием `find`.

Выполнение лабораторной работы

1. Поиск строк с использованием `getopts` и `grep`

Для первого задания использован командный файл `search.sh`, который обрабатывает параметры командной строки и выполняет поиск заданного шаблона в указанном файле.

Работа скрипта была проверена на тестовом текстовом файле.

Скрипт корректно обрабатывает переданные параметры и выполняет поиск средствами `grep`.

```

1  #!/bin/bash
2
3  input_file=""
4  output_file=""
5  pattern=""
6  case_sensitive=""
7  show_numbers=""
8
9  usage() {
10     echo "Использование: $0 [опции]"
11     echo "  -i <файл>    входной файл"
12     echo "  -o <файл>    выходной файл"
13     echo "  -p <шаблон>   шаблон для поиска"
14     echo "  -C           учитывать регистр (по умолчанию не учитывает)"
15     echo "  -n           показывать номера строк"
16     echo "  -h           показать эту справку"
17     exit 0
18 }
19
20 while getopts "i:op:Cnh" opt; do
21     case $opt in
22         i) input_file="$OPTARG" ;;
23         o) output_file="$OPTARG" ;;
24         p) pattern="$OPTARG" ;;
25         C) case_sensitive="-C" ;;
26         n) show_numbers="-n" ;;
27         h) usage ;;
28         *) echo "Неизвестная опция: '$OPTARG'"; usage ;;
29     esac
30 done
31
32 if [ -z "$input_file" ] || [ -z "$pattern" ]; then
33     echo "Ошибка: необходимо указать -i и -p"
34     usage
35 fi
36
37 if [ ! -f "$input_file" ]; then
38     echo "Ошибка: файл $input_file не найден"
39     exit 1
40 fi
41

```

Рис. 1: Листинг командного файла search.sh

```

Поиск шаблона 'test' в файле test.txt...
5:test line
mikhailov@fedora:~/reports/lab1$

```

Рис. 2: Результат выполнения search.sh

2. Определение знака числа и анализ кода завершения

Во втором задании создана программа `sign.c`. Она определяет знак введённого числа и завершает работу с определённым кодом возврата.

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 int main() {
5     int num;
6     printf("Введите число: ");
7     scanf("%d", &num);
8
9     if (num > 0) {
10        printf("Число %d больше нуля\n", num);
11        exit(1);
12    } else if (num < 0) {
13        printf("Число %d меньше нуля\n", num);
14        exit(2);
15    } else {
16        printf("Число равно нулю\n");
17        exit(0);
18    }
19 }
```

nichailvzvarov@fedora:~/reports/lab1\$

Рис. 3: Листинг программы `sign.c`

Для запуска программы и анализа кода завершения используется командный файл `check_sign.sh`. После выполнения С-программы скрипт анализирует специальную переменную `$?` и выводит соответствующее сообщение.

```
1 #!/bin/bash
2
3 ./sign
4 exit_code=$?
5
6 case $exit_code in
7     0) echo "Было введено число, равное нулю" ;;
8     1) echo "Было введено положительное число" ;;
9     2) echo "Было введено отрицательное число" ;;
10    *) echo "Неизвестный код завершения: $exit_code" ;;
11 esac
```

nichailvzvarov@fedora:~/reports/lab1\$

Рис. 4: Листинг командного файла `check_sign.sh`

Работа программы была проверена на тестовом значении.

```
Введите число: 2
Число 2 больше нуля
Было введено положительное число
michailvzarov@fedora:~/reports/lab1$
```

Рис. 5: Результат выполнения check_sign.sh

Полученный результат показывает корректную передачу информации через код завершения программы.

3. Создание и удаление временных файлов

В третьем задании реализован командный файл create_files.sh. Он позволяет создавать заданное количество файлов с последовательной нумерацией и удалять ранее созданные файлы.

Для проверки режима создания использован ключ -c с указанием количества файлов.

В результате были созданы файлы 1.tmp, 2.tmp, 3.tmp, 4.tmp и 5.tmp.

Затем был проверен режим удаления с помощью ключа -d.

После выполнения команды созданные временные файлы были удалены.

4. Архивирование файлов каталога

В четвёртом задании использован командный файл archive_recent.sh. Скрипт применяет команды find и tar для отбора и архивирования файлов.

Работа скрипта была проверена на каталоге лабораторной работы.

Архивирование выполняется корректно, а для отбора недавно изменённых файлов применяется команда find.

```

1 #!/bin/bash
2
3 usage() {
4     echo "Использование: $0 [опции]"
5     echo "  -c <N>   создать N файлов (1.tmp, 2.tmp, ...)"
6     echo "  -d       удалить все созданные файлы"
7     echo "  -h       показать эту справку"
8     exit 0
9 }
10
11 create_files() {
12     local n=${1:-5}
13     echo "Создание $n файлов..."
14     for i in $(seq 1 $n); do
15         touch "$i.tmp"
16         echo "Создан файл $i.tmp"
17     done
18     echo "Создано $n файлов"
19 }
20
21 delete_files() {
22     echo "Удаление файлов..."
23     for i in $(seq 1 100); do
24         if [ -f "$i.tmp" ]; then
25             rm "$i.tmp"
26             echo "Удалён файл $i.tmp"
27         fi
28     done
29     echo "Удаление завершено"
30 }
31
32 if [ $# -eq 0 ]; then
33     usage
34 fi
35
36 case "$1" in
37     -c)
38         if [ -z "$2" ] || [ "$2" -le 0 ]; then
39             echo "Ошибка: укажите положительное число"
40             usage
41         fi

```

Рис. 6: Листинг командного файла create_files.sh

```

Создание 5 файлов...
Создан файл 1.tmp
Создан файл 2.tmp
Создан файл 3.tmp
Создан файл 4.tmp
Создан файл 5.tmp
Создано 5 файлов

=== TMP FILES ===
./1.tmp
./2.tmp
./3.tmp
./4.tmp
./5.tmp
michailvzvarov@fedora:~/reports/lab13$

```

Рис. 7: Создание временных файлов

```

Удаление файлов...
Удален файл 1.tmp
Удален файл 2.tmp
Удален файл 3.tmp
Удален файл 4.tmp
Удален файл 5.tmp
Удаление завершено

=== TMP FILES AFTER DELETE ===
michailvzvarov@fedora:~/reports/lab13$

```

Рис. 8: Удаление временных файлов

```

1 #!/bin/bash
2
3
4 usage() {
5     echo "Использование: $0 <директория>"
6     echo "Архивирует все файлы в директории, изменённые менее 7 дней назад"
7     exit 0
8 }
9
10 if [ $# -ne 1 ]; then
11     usage
12 fi
13
14 DIR="$1"
15
16 if [ ! -d "$DIR" ]; then
17     echo "Ошибка: $DIR не является директорией"
18     exit 1
19 fi
20
21 ARCHIVE_NAME="archive_$(date +%Y%m%d_%H%M%S).tar.gz"
22
23 echo "Поиск файлов, изменённых менее 7 дней назад в $DIR..."
24
25 find "$DIR" -type f -mtime -7 -print0 | tar -czf "$ARCHIVE_NAME" --null -T -
26
27 echo "Архив создан: $ARCHIVE_NAME"
28 echo "Содержимое архива:"
29 tar -tzf "$ARCHIVE_NAME" | head -10
michailvzvarov@fedora:~/reports/lab13$

```

Рис. 9: Листинг командного файла archive_recent.sh

```
Использование: ./archive_recent.sh <директория>
Архивирует все файлы в директории, изменённые менее 7 дней назад

=== CREATED ARCHIVES ===
./archive_20200904_214959.tar.gz
./archive_20200904_215017.tar.gz
./archive_20200904_221031.tar.gz
mikhailvzvarov@fedora:~/reports/lab13$
```

Рис. 10: Результат выполнения archive_recent.sh

Вывод

В ходе лабораторной работы были изучены ветвления, циклы и обработка параметров командной строки в Bash. Были практически применены команды getopts, grep, find и tar, а также специальный параметр \$?. Была реализована передача информации о результате работы программы на языке C через код завершения. Кроме того, были созданы сценарии для создания и удаления временных файлов и архивирования недавно изменённых файлов.

Контрольные вопросы

1. Каково предназначение команды getopts?

Команда getopts предназначена для разбора параметров командной строки в shell-скриптах. Она позволяет обрабатывать короткие ключи и аргументы этих ключей.

2. Какое отношение метасимволы имеют к генерации имён файлов?

Метасимволы используются для шаблонной подстановки имён файлов. Символ * соответствует произвольной последовательности символов, ? — одному символу, а квадратные скобки задают набор или диапазон допустимых символов.

3. Какие операторы управления действиями вы знаете?

К основным управляющим конструкциям Bash относятся if, then, elif, else, fi, case, for, while, until, break и continue.

4. Какие операторы используются для прерывания цикла?

Для полного прерывания цикла используется break. Оператор continue прерывает только текущую итерацию и переходит к следующей.

5. Для чего нужны команды false и true?

Команда true всегда завершается с кодом 0, а false — с ненулевым кодом. Они применяются в логических выражениях, циклах и условных конструкциях.

6. Что означает строка if test -f mans/i.\$s?

Она проверяет, существует ли объект mans/i.\$s и является ли он обычным файлом. Ключ -f команды test возвращает истинное значение для обычного файла.

7. Объясните различия между конструкциями while и until.

Цикл while выполняется, пока проверяемое условие истинно. Цикл until выполняется, пока условие ложно, и завершается, когда оно становится истинным.

Список литературы

1. Методические материалы к лабораторной работе «Программирование в командном процессоре ОС UNIX. Ветвления и циклы».
2. Документация Bash (man bash).
3. Справочная документация по командам grep, find, tar и getopts.